

SV9 - Digital Verification using SV & UVM

====Project====

Synchronous FIFO verification using SV

Name

Ibrahim Mohamed Mohamed Hanafy

-Original FIFO Design Code: (Buggy)

```
module FIFO(FIFO_if.DUT F_if);

localparam max_fifo_addr = $clog2(F_if.FIFO_DEPTH);

reg [F_if.FIFO_WIDTH-1:0] mem [F_if.FIFO_DEPTH-1:0];

reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;
reg [max_fifo_addr:0] count;

always @(posedge F_if.clk or negedge F_if.rst_n) begin
    if (!F_if.rst_n) begin
        wr_ptr <= 0; //BUG: wr_ack isn't reset correctly
    end
    //BUG: overflow, underflow not reset correctly
    else if (F_if.wr_en && (count < F_if.FIFO_DEPTH)) begin
        mem[wr_ptr] <= F_if.data_in;
        F_if.wr_ack <= 1;
        wr_ptr <= wr_ptr + 1;
    end
    else begin
        F_if.wr_ack <= 0;
        if (F_if.full & F_if.wr_en)
            F_if.overflow <= 1;
        else
            F_if.overflow <= 0;
    end
end
end

always @(posedge F_if.clk or negedge F_if.rst_n) begin
    if (!F_if.rst_n) begin //BUG: data out isn't reset correctly
        rd_ptr <= 0;
    end
    else if (F_if.rd_en && count != 0) begin
        F_if.data_out <= mem[rd_ptr];
        rd_ptr <= rd_ptr + 1;
    end
end
end

always @(posedge F_if.clk or negedge F_if.rst_n) begin
    if (!F_if.rst_n) begin
        count <= 0;
    end
    else begin
        if ( ({F_if.wr_en, F_if.rd_en} == 2'b10) && !F_if.full)
            count <= count + 1;
        else if ( ({F_if.wr_en, F_if.rd_en} == 2'b01) && !F_if.empty)
            count <= count - 1; //BUG: count isn't updated when both read and write
    end
end
end

assign F_if.full = (count == F_if.FIFO_DEPTH)? 1 : 0;
assign F_if.empty = (count == 0)? 1 : 0;
assign F_if.underflow = (F_if.empty && F_if.rd_en)? 1 : 0; //BUG: underflow is
assign F_if.almostfull = (count == F_if.FIFO_DEPTH-2)? 1 : 0; //BUG : almostfull
assign F_if.almostempty = (count == 1)? 1 : 0;
```

-Verification Plan

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
Reset	When the rst_n is asserted, internal counters and pointers are reset to 0, leading to an empty FIFO	Directed at the start of the simulation, then randomized such that it is off most of the time	-	A check_data function in a scoreboard class, that checks against a reference_model. -> assertion check: a_count -> assertion check: a_pointers
Writing 1	When the wr_en is asserted, the data_in is written to an empty entry, if available.	Randomization, ST: It is active for a user set amount of time	Cover all cases with rd_en and each o/p control signal	A check_data function in a scoreboard class, that checks against a reference_model.
Writing 2	If writing was successful, wr_ack conveys acknowledgment of the writing process, and vice versa	"" ""	"" ""	-> assertion check: a_wr_ack_1 (ack) -> assertion check: a_wr_ack_2 (no ack)
Reading	When the rd_en is asserted, the data_out is read from an occupant entry, if available. The data read's entry is now free for another	Randomization, ST: It is active for a user set amount of time	Cover all cases with wr_en and each o/p control signal	A check_data function in a scoreboard class, that checks against a reference_model.
Writing and Reading	Reading has priority for full FIFO Writing has priority for an empty FIFO Simultaneous for a normal FIFO	Randomization, ST: Both are active for a user set amount of time	Cover all cases with wr_en and rd_en	A check_data function in a scoreboard class, that checks against a reference_model.
Overflowing	If the FIFO is full, and wr_en was asserted, the writing process is ignored, and the elements don't change, and a flag is raised	Randomization, ST: Writing is more probable than reading	Cover all cases with wr_en and rd_en	A check_data function in a scoreboard class, that checks against a reference_model. -> assertion check: a_overflow
Underflowing	If the FIFO is empty, and rd_en was asserted, the reading process is ignored, and the elements don't change, and a flag is raised	Randomization, ST: Reading is more probable than reading	Cover all cases with wr_en and rd_en	A check_data function in a scoreboard class, that checks against a reference_model. -> assertion check: a_underflow
Full	If the wr_en is asserted, and there all entries have been written to and not read yet, a full flag is raised	Randomization, ST: Writing is more probable than reading	Cover all cases with wr_en and rd_en	A check_data function in a scoreboard class, that checks against a reference_model. -> assertion check: a_full
Empty	If the rd_en is asserted, and there is no entries that were written to, an empty flag is raised	Randomization, ST: Reading is more probable than reading	Cover all cases with wr_en and rd_en	A check_data function in a scoreboard class, that checks against a reference_model. -> assertion check: a_empty
Almost Full	If there is only one entry remaining for writing, the almost full flag is raised	Randomization, ST: Writing is more probable than reading	Cover all cases with wr_en and rd_en	A check_data function in a scoreboard class, that checks against a reference_model. -> assertion check: a_almost_full
Almost Empty	If there is only one entry remaining for writing, the almost empty flag is raised	Randomization, ST: Reading is more probable than reading	Cover all cases with wr_en and rd_en	A check_data function in a scoreboard class, that checks against a reference_model. -> assertion check: a_almost_empty
Read Pointer Threshold	The Read pointer threshold value = depth of FIFO (cant ever surpass it)	Randomization	-	-> assertion check: a_rd_ptr_threshold
Read Pointer Threshold	The Write pointer threshold value = depth of FIFO (cant ever surpass it)	Randomization	-	-> assertion check: a_wr_ptr_threshold
Read Pointer wrapping	The read pointer wraps around after reaching its threshold value	Randomization	-	-> assertion check: a_rd_ptr_wrap
Write Pointer wrapping	The write pointer wraps around after reaching its threshold value	Randomization	-	-> assertion check: a_wr_ptr_wrap

- Shared Package:

```
package shared_pkg;

    class Test_variables;
        static integer correct_count = 0;
        static integer error_count = 0;
        static bit test_finished = 0;

        static event driving_done_by_tb;
        static event dut_is_done;
    endclass

endpackage
```

- Transaction Pac

```
package Transaction_pkg;

    class FIFO_Transaction #(parameter FIFO_WIDTH = 16);

        rand logic [FIFO_WIDTH-1:0] data_in;
        rand logic rst_n, wr_en, rd_en;

        logic [FIFO_WIDTH-1:0] data_out;
        logic wr_ack, overflow;
        logic full, empty, almostfull, almostempty, underflow;

        integer RD_EN_ON_DIST, WR_EN_ON_DIST;

        function new(input WR_EN_ON_DIST = 70, input RD_EN_ON_DIST = 30);
            this.WR_EN_ON_DIST = WR_EN_ON_DIST;
            this.RD_EN_ON_DIST = RD_EN_ON_DIST;
        endfunction

        //constraints=====
        constraint Reset { //off for 90% of the time
            rst_n dist {1:/90 , 0:/10};
        }
        constraint Write_en { //on for 70% of the time
            wr_en dist {1:/WR_EN_ON_DIST , 0:/(100-WR_EN_ON_DIST)};
        }
        constraint Read_en { //on for 30% of the time
            rd_en dist {1:/RD_EN_ON_DIST , 0:/(100-RD_EN_ON_DIST)};
        }

    endclass

endpackage
```

- Scoreboard Package:

```
package Scoreboard_pkg;

import Transaction_pkg::*;
import shared_pkg::*;

class FIFO_Scoreboard #(parameter FIFO_WIDTH = 16, parameter FIFO_DEPTH = 8); //need to override with

    logic [FIFO_WIDTH-1:0] data_out_ref;
    FIFO_Transaction #(FIFO_WIDTH(FIFO_WIDTH)) full_ref;

    logic [FIFO_WIDTH-1:0] fifo_mem_q[$];

    function automatic void check_data(input FIFO_Transaction #(FIFO_WIDTH(FIFO_WIDTH)) F_txn);

        full_ref = new();
        reference_model(F_txn);

        if(something_is_wrong(F_txn)) begin
            Test_variables::error_count ++;
            $display($sformatf("Error @ %0t: \n", $time),
                $sformatf("Stimulii: wr_en=%b, rd_en=%b, data_in=0x%0h, rst_n=%b\n", F_txn.wr_en, F_txn.
                $sformatf("Expected: data_out=0x%0h | full=%b | empty=%b | wr_ack=%b | overflow=%b | und
                $sformatf("Actual:   data_out=0x%0h | full=%b | empty=%b | wr_ack=%b | overflow=%b | und
                $sformatf("=====
            );
        end else begin
            Test_variables::correct_count ++;
        end
    endfunction

function new();
    full_ref = new();
    data_out_ref = {FIFO_WIDTH{1'b0}};
endfunction

function automatic bit something_is_wrong(input FIFO_Transaction
#(FIFO_WIDTH(FIFO_WIDTH)) F_txn);
    return (
        F_txn.wr_ack      != full_ref.wr_ack      ||
        F_txn.overflow    != full_ref.overflow    ||
        F_txn.full        != full_ref.full        ||
        F_txn.empty        != full_ref.empty        ||
        F_txn.underflow    != full_ref.underflow    ||
        F_txn.almostfull    != full_ref.almostfull ||
        F_txn.almostempty    != full_ref.almostempty ||
        F_txn.data_out      != data_out_ref
    );
endfunction
```

```
function new();
    full_ref = new();
    data_out_ref = {FIFO_WIDTH{1'b0}};
endfunction

function automatic bit something_is_wrong(input FIFO_Transaction
#(FIFO_WIDTH(FIFO_WIDTH)) F_txn);
    return (
        F_txn.wr_ack      != full_ref.wr_ack      ||
        F_txn.overflow    != full_ref.overflow    ||
        F_txn.full        != full_ref.full        ||
        F_txn.empty        != full_ref.empty        ||
        F_txn.underflow    != full_ref.underflow    ||
        F_txn.almostfull    != full_ref.almostfull ||
        F_txn.almostempty    != full_ref.almostempty ||
        F_txn.data_out      != data_out_ref
    );
endfunction
```

```
function automatic void reference_model(input FIFO_Transac
#(FIFO_WIDTH(FIFO_WIDTH)) F_txn);

    // Default pulse-like sequential outputs every cycle
    full_ref.wr_ack      = 0;
    full_ref.overflow    = 0;
    full_ref.underflow    = 0;

    if (!F_txn.rst_n) begin
        fifo_mem_q.delete();
        data_out_ref      = {FIFO_WIDTH{1'b0}};

        full_ref.full      = 0;
        full_ref.empty      = 1;
        full_ref.almostfull = 0;
        full_ref.almostempty = 0;
        full_ref.wr_ack      = 0;
        full_ref.overflow    = 0;
        full_ref.underflow    = 0;

        full_ref.data_out = data_out_ref;
    end
    else begin
        // Operation handling
        // -----
        case ({F_txn.wr_en, F_txn.rd_en})

            2'b10: begin
                // Write only
                if (fifo_mem_q.size() < FIFO_DEPTH) begin
                    fifo_mem_q.push_back(F_txn.data_in);
                    full_ref.wr_ack = 1;
                end
            else begin
                full_ref.overflow = 1;
            end
        end
    end
end
```

```
2'b01: begin
    // Read only
    if (fifo_mem_q.size() > 0) begin
        data_out_ref = fifo_mem_q.pop_front();
    end
    else begin
        full_ref.underflow = 1;
    end
end

2'b11: begin
    // Both write and read enabled
    if (fifo_mem_q.size() == 0) begin
        // Spec note: when empty, only writing takes
        fifo_mem_q.push_back(F_txn.data_in);
        full_ref.wr_ack = 1;
    end
    else if (fifo_mem_q.size() == FIFO_DEPTH) begin
        // Spec note: when full, only reading takes
        data_out_ref = fifo_mem_q.pop_front();
    end
    else begin
        // Normal simultaneous read + write
        data_out_ref = fifo_mem_q.pop_front();
        fifo_mem_q.push_back(F_txn.data_in);
        full_ref.wr_ack = 1;
    end
end

default: begin
    // No operation

    // -----
    // Combinational flag generation from resulting occupancy
    // -----
    full_ref.full      = (fifo_mem_q.size() == FIFO_DEPTH);
    full_ref.empty      = (fifo_mem_q.size() == 0);
    full_ref.almostfull = (fifo_mem_q.size() == FIFO_DEPTH-1);
    full_ref.almostempty = (fifo_mem_q.size() == 1);

    // Optional if your transaction object has data_out
    full_ref.data_out    = data_out_ref;
end
```

- Coverage Package:

```
package Coverage_pkg;

import Transaction_pkg::*;

class FIFO_Coverage #(parameter FIFO_WIDTH = 16);

    FIFO_Transaction #(.FIFO_WIDTH(FIFO_WIDTH)) F_cvg_txn;

    //Coverpoints and bins=====
    covergroup CovPort; //@(posedge clk)
        rst_cp: coverpoint F_cvg_txn.rst_n iff(1){}
        wr_en_cp: coverpoint F_cvg_txn.wr_en iff(1){}
        rd_en_cp: coverpoint F_cvg_txn.rd_en iff(1){}

        wr_ack_cp: coverpoint F_cvg_txn.wr_ack iff(1){}
        OF_cp: coverpoint F_cvg_txn.overflow iff(1){}
        UF_cp: coverpoint F_cvg_txn.underflow iff(1){}
        Full_cp: coverpoint F_cvg_txn.full iff(1){}
        Empty_cp: coverpoint F_cvg_txn.empty iff(1){}
        almost_full_cp: coverpoint F_cvg_txn.almostfull iff(1){}
        almost_empty_cp: coverpoint F_cvg_txn.almostempty iff(1){}
    endcovergroup

function void sample_data(input FIFO_Transaction F_txn);
    #(.FIFO_WIDTH(FIFO_WIDTH)) F_txn;
    F_cvg_txn = F_txn;
    CovPort.sample();
endfunction

function new();
    CovPort = new();
    F_cvg_txn = new();
endfunction

//1.
wr_rd_wr_ack: cross wr_en_cp, rd_en_cp, wr_ack_cp {
    ignore_bins impossible =
        binsof(wr_en_cp) intersect {0} &&
        binsof(wr_ack_cp) intersect {1};}

//2.
wr_rd_OF: cross wr_en_cp, rd_en_cp, OF_cp {
    ignore_bins impossible =
        (binsof(wr_en_cp) intersect {0} &&
        binsof(OF_cp) intersect {1})
        ||
        (binsof(wr_en_cp) intersect {1} &&
        binsof(rd_en_cp) intersect {1} &&
        binsof(OF_cp) intersect {1});}

//3.
wr_rd_UF: cross wr_en_cp, rd_en_cp, UF_cp {
    ignore_bins impossible =
        (binsof(rd_en_cp) intersect {0} &&
        binsof(UF_cp) intersect {1})
        ||
        (binsof(wr_en_cp) intersect {1} &&
        binsof(rd_en_cp) intersect {1} &&
        binsof(UF_cp) intersect {1});}

//4.
wr_rd_Full: cross wr_en_cp, rd_en_cp, Full_cp {
    ignore_bins impossible =
        binsof(rd_en_cp) intersect {1} &&
        binsof(Full_cp) intersect {1};
}

//5.
wr_rd_Empty: cross wr_en_cp, rd_en_cp, Empty_cp{}

//6.
wr_rd_almost_full: cross wr_en_cp, rd_en_cp, almost_full_cp{}

//7.
wr_rd_almost_empty: cross wr_en_cp, rd_en_cp, almost_empty_cp{}
}
```

- Testbench:

```
module FIFO_tb(FIFO_if.TEST F_if);

import shared_pkg::*;

import Transaction_pkg::*;
import Coverage_pkg::*;
import Scoreboard_pkg::*;

    FIFO_Transaction #(FIFO_WIDTH(F_if.FIFO_WIDTH)) F_txn = new();

initial begin
    Test_variables::error_count = 0;
    Test_variables::correct_count = 0;
    Test_variables::test_finished = 0;

    // Initialize transaction object
    F_txn.rst_n = 1;
    F_txn.wr_en = 0;
    F_txn.rd_en = 0;
    F_txn.data_in = '0;
    drive_FIFO_if_with_txn_stimulii();

    //Test Series 1: Reset
    assert reset();

    //Test Series 2: Random Stimulii with write more probable than read
    repeat(10000) begin
        @(negedge F_if.clk);
        #1; //wait for monitor to be done before driving
        F_txn.RD_EN_ON_DIST = 30;
        F_txn.WR_EN_ON_DIST = 70;
        assert(F_txn.randomize()) else $fatal("Randomization failed");
        drive_FIFO_if_with_txn_stimulii();

        @(negedge F_if.clk);
        -> Test_variables::driving_done_by_tb; //trigger event for monitor
    end

    //Test Series 3: Random Stimulii with read more probable than write
    repeat(10000) begin
        @(negedge F_if.clk);
        #1;
        F_txn.RD_EN_ON_DIST = 70;
        F_txn.WR_EN_ON_DIST = 30;
        assert(F_txn.randomize()) else $fatal("Randomization failed");
        drive_FIFO_if_with_txn_stimulii();

        @(negedge F_if.clk);
        -> Test_variables::driving_done_by_tb; //trigger event for monitor
    end

    Test_variables::test_finished = 1;
end

function automatic void drive_FIFO_if_with_txn_stimulii();
    F_if.rst_n = F_txn.rst_n;
    F_if.wr_en = F_txn.wr_en;
    F_if.rd_en = F_txn.rd_en;
    F_if.data_in = F_txn.data_in;
endfunction

task automatic assert_reset();
    F_txn.rst_n = 0;
    drive_FIFO_if_with_txn_stimulii();
    #1;
    -> Test_variables::driving_done_by_tb;
    F_txn.rst_n = 1;
    drive_FIFO_if_with_txn_stimulii();
endtask
```


- Monitor:

```
module FIFO_monitor(FIFO_if.MONITOR F_if);

import shared_pkg::*;

import Transaction_pkg::*;
import Coverage_pkg::*;
import Scoreboard_pkg::*;

FIFO_Transaction #(FIFO_WIDTH(F_if.FIFO_WIDTH)) F_txn = new();
FIFO_Coverage #(FIFO_WIDTH(F_if.FIFO_WIDTH)) F_cov = new();
FIFO_Scoreboard #(FIFO_WIDTH(F_if.FIFO_WIDTH)
, FIFO_DEPTH(F_if.FIFO_DEPTH)) F_sb = new();

initial begin
    forever begin
        if(Test_variables::test_finished) begin
            $display("Simulation finished with %0d errors and %0d c
            $stop;
        end

        @(Test_variables::driving_done_by_tb); //wait for event tri
        assign_stimulii_FIFO_if_to_txn();

        assign_response_FIFO_if_to_txn();
        fork
            begin //Thread 1: Coverage coll
                F_cov.sample_data(F_txn);
            end

            begin //Thread 2: Scoreboard cl
                F_sb.check_data(F_txn);
            end

        join //wait for both threads

    end
end
```

```
function void assign_stimulii_FIFO_if_to_txn();
    F_txn.rst_n = F_if.rst_n;
    F_txn.wr_en = F_if.wr_en;
    F_txn.rd_en = F_if.rd_en;
    F_txn.data_in = F_if.data_in;
endfunction

function void assign_response_FIFO_if_to_txn();
    F_txn.data_out = F_if.data_out;
    F_txn.wr_ack = F_if.wr_ack;
    F_txn.overflow = F_if.overflow;
    F_txn.underflow = F_if.underflow;
    F_txn.full = F_if.full;
    F_txn.empty = F_if.empty;
    F_txn.almostfull = F_if.almostfull;
    F_txn.almostempty = F_if.almostempty;
endfunction
```


- Assertions:

```
//combinational properties-----

property p_count_range;
  @(posedge F_if.clk) disable iff (!F_if.rst_n)
    (count >= 0) && (count <= F_if.FIFO_DEPTH);
endproperty

property p_full;
  @(posedge F_if.clk) disable iff (!F_if.rst_n)
    (F_if.full) |-> (count == F_if.FIFO_DEPTH);
endproperty

property p_empty;
  @(posedge F_if.clk) disable iff (!F_if.rst_n)
    (F_if.empty) |-> (count == 0);
endproperty

property p_almost_full;
  @(posedge F_if.clk) disable iff (!F_if.rst_n)
    (F_if.almostfull) |-> (count == (F_if.FIFO_DEPTH - 1));
endproperty

property p_almost_empty;
  @(posedge F_if.clk) disable iff (!F_if.rst_n)
    (F_if.almostempty) |-> (count == 1);
endproperty
```

```
property p_wr_ack_1; //(sequential)-----
  @(posedge F_if.clk) disable iff (!F_if.rst_n)
    (F_if.wr_en && !F_if.full) | => (F_if.wr_ack == 1);
endproperty

property p_wr_ack_2;
  @(posedge F_if.clk) disable iff (!F_if.rst_n)
    (F_if.wr_en && F_if.full) | => (F_if.wr_ack == 0);
endproperty

property p_rd_ptr_wrap;
  @(posedge F_if.clk) disable iff (!F_if.rst_n)
    (F_if.rd_en &&
      !F_if.empty &&
      !(F_if.wr_en && F_if.empty) &&
      (rd_ptr == F_if.FIFO_DEPTH-1))
      | => (rd_ptr == 0);
endproperty

property p_wr_ptr_wrap;
  @(posedge F_if.clk) disable iff (!F_if.
    (F_if.wr_en &&
      !F_if.full &&
      !(F_if.rd_en && F_if.full) &&
      (wr_ptr == F_if.FIFO_DEPTH-1))
      | => (wr_ptr == 0);
endproperty
```

```
always_comb begin
  if(!F_if.rst_n) begin
    a_reset: assert final(count == 0);
    a_rd_ptr_reset: assert final(rd_ptr == 0);
    a_wr_ptr_reset: assert final(wr_ptr == 0);
    a_wr_ack_reset: assert final(F_if.wr_ack == 0);
    a_empty_reset: assert final(F_if.empty == 1);
    a_full_reset: assert final(F_if.full == 0);
    a_almostfull_reset: assert final(F_if.almostfull == 0);
    a_almostempty_reset: assert final(F_if.almostempty == 0);
    a_overflow_reset: assert final(F_if.overflow == 0);
    a_underflow_reset: assert final(F_if.underflow == 0);
    a_data_out_reset: assert final(F_if.data_out == 0);
  end
end
```

```
property p_v
  @(posedge
    wr_ptr < end
endproperty

property p_rd_ptr_threshold;
  @(posedge F_if.clk) disable iff (!F_if.rst_n)
    rd_ptr < F_if.FIFO_DEPTH;
endproperty

property p_overflow;
  @(posedge F_if.clk) disable iff (!F_if.rst_n)
    (F_if.wr_en && F_if.full && !F_if.rd_en) | => (F_if.overflow
endproperty

property p_underflow;
  @(posedge F_if.clk) disable iff (!F_if.rst_n)
    (F_if.rd_en && F_if.empty && !F_if.wr_en) | => (F_if.underflo
endproperty
```

- Top

```
module FIFO_top();
// 1. Generate the clock
bit clk;
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

// 2. instantiate the interface, and pass the clock
FIFO_if f_if(clk);

// 3. instantiate the tb, DUT, monitor, and pass the
FIFO dut (f_if);

FIFO_tb tb (f_if);

FIFO_monitor monitor (f_if);

endmodule
```

- Do file

```
#vlib work
vlog -sv -f ../../VS_code/Project_FIFO/Enviroment/src_files.list +cover -covercells
vsim -voptargs=+acc work.FIFO_top -cover
#add wave /top/ALSU_if/*
coverage save FIFO_top.ucdb -onexit
do wave.do
run -all

vcover report FIFO_top.ucdb -details -all -output code_cvr.txt
#-sv_seed random -l sim.log //at end of vsim
```

1st Results

-Assertions

+ /FIFO_top/dut/a_wr_ack_1	Concurrent	SVA	on	0	1
+ /FIFO_top/dut/a_wr_ack_2	Concurrent	SVA	on	0	1
+ /FIFO_top/dut/a_rd_ptr_wrap	Concurrent	SVA	on	0	1
+ /FIFO_top/dut/a_wr_ptr_wrap	Concurrent	SVA	on	0	1
+ /FIFO_top/dut/a_overflow	Concurrent	SVA	on	0	1
+ /FIFO_top/dut/a_underflow	Concurrent	SVA	on	1111	1
+ /FIFO_top/dut/a_full	Concurrent	SVA	on	0	1
+ /FIFO_top/dut/a_empty	Concurrent	SVA	on	0	1
+ /FIFO_top/dut/a_almost_full	Concurrent	SVA	on	442	0
+ /FIFO_top/dut/a_almost_empty	Concurrent	SVA	on	0	1
+ /FIFO_top/dut/a_wr_ptr_threshold	Concurrent	SVA	on	0	1
+ /FIFO_top/dut/a_rd_ptr_threshold	Concurrent	SVA	on	0	1
+ /FIFO_top/dut/a_count_range	Concurrent	SVA	on	0	1
+ /FIFO_top/dut/a_reset	Immediate	SVA	on	0	1
+ /FIFO_top/dut/a_rd_ptr_reset	Immediate	SVA	on	0	1
+ /FIFO_top/dut/a_wr_ptr_reset	Immediate	SVA	on	0	1
+ /FIFO_top/dut/a_wr_ack_reset	Immediate	SVA	on	866	1
+ /FIFO_top/dut/a_empty_reset	Immediate	SVA	on	0	1
+ /FIFO_top/dut/a_full_reset	Immediate	SVA	on	0	1
+ /FIFO_top/dut/a_almostfull_reset	Immediate	SVA	on	0	1
+ /FIFO_top/dut/a_almostempty_reset	Immediate	SVA	on	0	1
+ /FIFO_top/dut/a_overflow_reset	Immediate	SVA	on	112	1
+ /FIFO_top/dut/a_underflow_reset	Immediate	SVA	on	979	1
+ /FIFO_top/dut/a_data_out_reset	Immediate	SVA	on	1901	0
+ /FIFO_top/tb/#ublk#182146786#33/immed_38	Immediate	SVA	on	0	1
+ /FIFO_top/tb/#ublk#182146786#47/immed_52	Immediate	SVA	on	0	1

-Some scoreboard errors

```
# Error @ 199980:
# Stimulii: wr_en=1, rd_en=1, data_in=0xl8ef, rst_n=1
# Expected: data_out=0xl8ef | full=0 | empty=1 | wr_ack=1 | overflow=0 | underflow=0 | almostfull=0 | almostempty=1
# Actual:   data_out=0x8el3 | full=0 | empty=1 | wr_ack=1 | overflow=0 | underflow=1 | almostfull=0 | almostempty=0
# =====
# Error @ 199990:
# Stimulii: wr_en=0, rd_en=1, data_in=0x5e95, rst_n=1
# Expected: data_out=0xl8ef | full=0 | empty=1 | wr_ack=0 | overflow=0 | underflow=0 | almostfull=0 | almostempty=0
# Actual:   data_out=0x8el3 | full=0 | empty=1 | wr_ack=0 | overflow=0 | underflow=1 | almostfull=0 | almostempty=0
# =====
# Error @ 200000:
# Stimulii: wr_en=0, rd_en=1, data_in=0x7c22, rst_n=1
# Expected: data_out=0xl8ef | full=0 | empty=1 | wr_ack=0 | overflow=0 | underflow=1 | almostfull=0 | almostempty=0
# Actual:   data_out=0x8el3 | full=0 | empty=1 | wr_ack=0 | overflow=0 | underflow=1 | almostfull=0 | almostempty=0
# =====
# Error @ 199900:
# Stimulii: wr_en=0, rd_en=1, data_in=0xel45, rst_n=1
# Expected: data_out=0x85be | full=0 | empty=1 | wr_ack=0 | overflow=0 | underflow=0 | almostfull=0 | almostempty=0
# Actual:   data_out=0xl07c | full=0 | empty=1 | wr_ack=0 | overflow=0 | underflow=1 | almostfull=0 | almostempty=0
# =====
# Error @ 199910:
# Stimulii: wr_en=0, rd_en=1, data_in=0xfeab, rst_n=1
# Expected: data_out=0x85be | full=0 | empty=1 | wr_ack=0 | overflow=0 | underflow=1 | almostfull=0 | almostempty=0
# Actual:   data_out=0xl07c | full=0 | empty=1 | wr_ack=0 | overflow=0 | underflow=1 | almostfull=0 | almostempty=0
# =====
# Error @ 199920:
# Stimulii: wr_en=0, rd_en=1, data_in=0xf4a1, rst_n=1
# Expected: data_out=0x85be | full=0 | empty=1 | wr_ack=0 | overflow=0 | underflow=1 | almostfull=0 | almostempty=0
# Actual:   data_out=0xl07c | full=0 | empty=1 | wr_ack=0 | overflow=0 | underflow=1 | almostfull=0 | almostempty=0
# =====
```

-Bug Hunting

Based on the assertions results, we can point out the following bugs found in the RTL code:

```
assign F_if.underflow = (F_if.empty && F_if.rd_en)? 1 : 0; //BUG: underflow is sequential
assign F_if.almostfull = (count == F_if.FIFO_DEPTH-2)? 1 : 0; //BUG : almostfull condition is wrong
```

-1:Underflow isn't sequential as specified in the specs

-2:Wrong almost full condition

```
always @(posedge F_if.clk or negedge F_if.rst_n) begin
    if (!F_if.rst_n) begin
        wr_ptr <= 0; //BUG: wr_ack isn't reset correctly
    end
    //BUG: overflow, underflow not reset correctly

always @(posedge F_if.clk or negedge F_if.rst_n) begin
    if (!F_if.rst_n) begin //BUG: data out isn't reset correctly
        rd_ptr <= 0;
    end
```

-3:Incorrect resetting behavior for: wr_ack, underflow, overflow, data_out

After fixing the earlier bugs, we can see the scoreboard convey a problem with the flags at the moment of simultaneous read and write

```
always @(posedge F_if.clk or negedge F_if.rst_n) begin
    if (!F_if.rst_n) begin
        count <= 0;
    end
    else begin
        if ( ({F_if.wr_en, F_if.rd_en} == 2'b10) && !F_if.full)
            count <= count + 1;
        else if ( ({F_if.wr_en, F_if.rd_en} == 2'b01) && !F_if.empty)
            count <= count - 1; //BUG: count isn't updated when both read and write are enabled
    end
end
```

-4:No handling of simultaneous read and write

```
else begin
    F_if.wr_ack <= 0;
    if (F_if.full & F_if.wr_en) //BUG: overflow condition is wrong
        F_if.overflow <= 1;
    else
        F_if.overflow <= 0;
end
```

-5:Incorrect conditions for overflow and underflow

-Finalized Design (Bug Free)

```
module FIFO(FIFO_if.DUT F_if);

localparam max_fifo_addr = $clog2(F_if.FIFO_DEPTH);

reg [F_if.FIFO_WIDTH-1:0] mem [F_if.FIFO_DEPTH-1:0];

reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;
reg [max_fifo_addr:0] count;

//Writing logic
always @(posedge F_if.clk or negedge F_if.rst_n) begin
    if (!F_if.rst_n) begin
        wr_ptr <= 0;
        F_if.wr_ack <= 0;
        F_if.overflow <= 0;
    end
    else if (F_if.wr_en && (count < F_if.FIFO_DEPTH)) begin
        mem[wr_ptr] <= F_if.data_in;
        F_if.wr_ack <= 1;
        wr_ptr <= wr_ptr + 1;
        F_if.overflow <= 0;
    end else begin
        F_if.wr_ack <= 0;
        wr_ptr <= wr_ptr;
        if (F_if.wr_en && (count == F_if.FIFO_DEPTH) && !F_if.rd_en)
            F_if.overflow <= 1;
        else
            F_if.overflow <= 0;
    end
end
end
```

```

//Reading logic
always @(posedge F_if.clk or negedge F_if.rst_n) begin
    if (!F_if.rst_n) begin
        rd_ptr <= 0;
        F_if.data_out <= 0;
        F_if.underflow <= 0;
    end
    else if (F_if.rd_en && count != 0) begin
        F_if.data_out <= mem[rd_ptr];
        rd_ptr <= rd_ptr + 1;
        F_if.underflow <= 0;
    end else begin
        if (F_if.rd_en && count == 0 && !F_if.wr_en)
            F_if.underflow <= 1;
        else
            F_if.underflow <= 0;
    end
end

always @(posedge F_if.clk or negedge F_if.rst_n) begin
    if (!F_if.rst_n) begin
        count <= 0;
    end
    else begin
        if ({F_if.wr_en, F_if.rd_en} == 2'b10) && !F_if.full)
            count <= count + 1;
        else if ({F_if.wr_en, F_if.rd_en} == 2'b01) && !F_if.empty)
            count <= count - 1;
        else if ({F_if.wr_en, F_if.rd_en} == 2'b11) begin
            if (F_if.empty == 1) //Writing if empty
                count <= count + 1;
            if (F_if.full == 1) //Reading if full
                count <= count - 1;
            //no race since FIFO cant be full and empty at the same time
        end
    end
end

assign F_if.full = (count == F_if.FIFO_DEPTH)? 1 : 0;
assign F_if.empty = (count == 0)? 1 : 0;
assign F_if.almostfull = (count == F_if.FIFO_DEPTH-1)? 1 : 0;
assign F_if.almostempty = (count == 1)? 1 : 0;

```


Final Results

-Assertions

Name	Assertion Type	Language	Enable	Failure Count	Pass Count
/FIFO_top/dut/a_wr_ack_1	Concurrent	SVA	on	0	1
/FIFO_top/dut/a_wr_ack_2	Concurrent	SVA	on	0	1
/FIFO_top/dut/a_rd_ptr_wrap	Concurrent	SVA	on	0	1
/FIFO_top/dut/a_wr_ptr_wrap	Concurrent	SVA	on	0	1
/FIFO_top/dut/a_overflow	Concurrent	SVA	on	0	1
/FIFO_top/dut/a_underflow	Concurrent	SVA	on	0	1
/FIFO_top/dut/a_full	Concurrent	SVA	on	0	1
/FIFO_top/dut/a_empty	Concurrent	SVA	on	0	1
/FIFO_top/dut/a_almost_full	Concurrent	SVA	on	0	1
/FIFO_top/dut/a_almost_empty	Concurrent	SVA	on	0	1
/FIFO_top/dut/a_wr_ptr_threshold	Concurrent	SVA	on	0	1
/FIFO_top/dut/a_rd_ptr_threshold	Concurrent	SVA	on	0	1
/FIFO_top/dut/a_count_range	Concurrent	SVA	on	0	1
/FIFO_top/dut/a_reset	Immediate	SVA	on	0	1
/FIFO_top/dut/a_rd_ptr_reset	Immediate	SVA	on	0	1
/FIFO_top/dut/a_wr_ptr_reset	Immediate	SVA	on	0	1
/FIFO_top/dut/a_wr_ack_reset	Immediate	SVA	on	0	1
/FIFO_top/dut/a_empty_reset	Immediate	SVA	on	0	1
/FIFO_top/dut/a_full_reset	Immediate	SVA	on	0	1
/FIFO_top/dut/a_almostfull_reset	Immediate	SVA	on	0	1
/FIFO_top/dut/a_almostempty_reset	Immediate	SVA	on	0	1
/FIFO_top/dut/a_overflow_reset	Immediate	SVA	on	0	1
/FIFO_top/dut/a_underflow_reset	Immediate	SVA	on	0	1
/FIFO_top/dut/a_data_out_reset	Immediate	SVA	on	0	1
/FIFO_top/tb/#ublk#182146786#33/immed__38	Immediate	SVA	on	0	1
/FIFO_top/tb/#ublk#182146786#47/immed__52	Immediate	SVA	on	0	1

/FIFO_top/dut/a_wr_ack_1...	SVA	✓	Off	7385	1	Unli...	1	100%		✓
/FIFO_top/dut/a_wr_ack_2...	SVA	✓	Off	664	1	Unli...	1	100%		✓
/FIFO_top/dut/a_rd_ptr_wra...	SVA	✓	Off	117	1	Unli...	1	100%		✓
/FIFO_top/dut/a_wr_ptr_wr...	SVA	✓	Off	395	1	Unli...	1	100%		✓
/FIFO_top/dut/a_overflow_c...	SVA	✓	Off	471	1	Unli...	1	100%		✓
/FIFO_top/dut/a_underflow_...	SVA	✓	Off	2660	1	Unli...	1	100%		✓
/FIFO_top/dut/a_full_cover	SVA	✓	Off	1078	1	Unli...	1	100%		✓
/FIFO_top/dut/a_empty_cov...	SVA	✓	Off	7213	1	Unli...	1	100%		✓
/FIFO_top/dut/a_almost_full...	SVA	✓	Off	845	1	Unli...	1	100%		✓
/FIFO_top/dut/a_almost_em...	SVA	✓	Off	4338	1	Unli...	1	100%		✓
/FIFO_top/dut/a_wr_ptr_thr...	SVA	✓	Off	17978	1	Unli...	1	100%		✓
/FIFO_top/dut/a_rd_ptr_thr...	SVA	✓	Off	17978	1	Unli...	1	100%		✓
/FIFO_top/dut/a_count_ran...	SVA	✓	Off	17978	1	Unli...	1	100%		✓

-Scoreboard results

```

# Loading WorkFile: monitor(1830)
# ** Warning: (vsim-WLF-5000) WLF file currently in use: vsim.wlf
#       File in use by: Ibrahim Hostname: HEMA ProcessID: 15304
#       Attempting to use alternate WLF file "./wlft4kqfri".
# ** Warning: (vsim-WLF-5001) Could not open WLF file: vsim.wlf
#       Using alternate file: ./wlft4kqfri
# Simulation finished with 0 errors and 20001 correct
# ** Note: $stop      : ../../VS_code/Project_FIFO/Environment/FIFO_monitor.sv(22)
#       Time: 200 us Iteration: 2 Instance: /FIFO_top/monitor
# Break in Module FIFO_monitor at ../../VS_code/Project_FIFO/Environment/FIFO_monitor.sv line 22
# QuestaSim-64 vcover 10.6c Coverage Utility 2017.07 Jul 26 2017

```

-Code Coverage

```
=====
```

=== File: ../../VS_code/Project_FIFO/Enviroment/FIFO.sv

```
=====
```

Statement Coverage:

Enabled Coverage	Active	Hits	Misses	% Covered
-----	-----	----	-----	-----
Stmts	22	22	0	100.0

Branch Coverage:

Enabled Coverage	Active	Hits	Misses	% Covered
-----	-----	----	-----	-----
Branches	25	25	0	100.0

Toggle Coverage:

Enabled Coverage	Active	Hits	Misses	% Covered
-----	-----	----	-----	-----
Toggle Bins	20	20	0	100.0

-Functional Coverage

TYPE CovPort	FIFO_Cover...	100.0%	100	100.0%		✓
+ CVP CovPort::rst_cp	FIFO_Cover...	100.0%	100	100.0%		✓
+ CVP CovPort::wr_en_cp	FIFO_Cover...	100.0%	100	100.0%		✓
+ CVP CovPort::rd_en_cp	FIFO_Cover...	100.0%	100	100.0%		✓
+ CVP CovPort::wr_ack_cp	FIFO_Cover...	100.0%	100	100.0%		✓
+ CVP CovPort::OF_cp	FIFO_Cover...	100.0%	100	100.0%		✓
+ CVP CovPort::UF_cp	FIFO_Cover...	100.0%	100	100.0%		✓
+ CVP CovPort::Full_cp	FIFO_Cover...	100.0%	100	100.0%		✓
+ CVP CovPort::Empty_cp	FIFO_Cover...	100.0%	100	100.0%		✓
+ CVP CovPort::almost_full_cp	FIFO_Cover...	100.0%	100	100.0%		✓
+ CVP CovPort::almost_empty_cp	FIFO_Cover...	100.0%	100	100.0%		✓
+ CROSS CovPort::wr_rd_wr_ack	FIFO_Cover...	100.0%	100	100.0%		✓
+ CROSS CovPort::wr_rd_OF	FIFO_Cover...	100.0%	100	100.0%		✓
+ CROSS CovPort::wr_rd_UF	FIFO_Cover...	100.0%	100	100.0%		✓
+ CROSS CovPort::wr_rd_Full	FIFO_Cover...	100.0%	100	100.0%		✓
+ CROSS CovPort::wr_rd_Empty	FIFO_Cover...	100.0%	100	100.0%		✓
+ CROSS CovPort::wr_rd_almost_full	FIFO_Cover...	100.0%	100	100.0%		✓
+ CROSS CovPort::wr_rd_almost_empty	FIFO_Cover...	100.0%	100	100.0%		✓

-Waveforms

